

Docker

Name: Viraj Patel

ID: 202611047

Step 1: Run your first container

Theory

A container is one program running on your computer, wrapped up so that it thinks it has the whole machine to itself. Instead of installing and configuring software directly on the host OS, Docker allows us to run isolated applications from pre-built images. Port mapping lets traffic from our host machine flow into the container.

Execution

I ran an `nginx:alpine` container without actually installing Nginx on my machine. I used the `-p 8080:80` flag to route local traffic to the container. The web server successfully served the default HTML page when accessed via `curl` and the browser.

A screenshot of a macOS desktop with a terminal window open. The terminal window has a title bar with standard macOS window controls (red, yellow, green buttons) and a title "viral -- ssh -- 289x75". The terminal content shows the output of the command "docker --help". The output is divided into sections for Client, Server, and a "Hello from Docker!" message. The Client section lists version, API version, OS version, Git commit, Build, OS/Arch, and Experimental status. The Server section lists engine version, API version, OS version, Git commit, Build, OS/Arch, Experimental status, containerd version, Git commit, runtime version, Git commit, Docker-Init version, and Docker-Init commit. The "Hello from Docker!" message states that the installation appears to be working correctly. Below this, there are instructions on how to generate this message, a list of steps to follow, and a list of Docker images available on the Docker Hub. The terminal also shows the command "docker run -it ubuntu bash" being executed. The desktop background is dark, and various application icons are visible at the bottom, including Safari, Photos, Google Chrome, App Store, Messages, WhatsApp, Mail, Calendar, TV, App Store, Settings, Calculator, Notes, and a trash can.

```
docker.io/library/registry:2
(base) viraj@Virajs-MacBook-Pro ~ % docker run -d --name web -p 8080:80 nginx:alpine
curl -s http://localhost:8080 | grep -i "<title>"

c15c641781057cce29f10f09f37226497914545b83a1e5c13f09815faaef9c4a
<title>Welcome to nginx!</title>
(base) viraj@Virajs-MacBook-Pro ~ %
```

Step 2: Images vs containers (Start and Stop)

Theory

An image is the recipe (a frozen, read-only snapshot of an app). A container is the meal (a running instance of that recipe). A critical distinction in Docker is that `docker stop` simply pauses and keeps the container, whereas `docker rm` deletes it forever.

Execution

I demonstrated container lifecycle management by stopping the running Nginx container using `docker stop`. I then verified that the website went offline, proving the container was halted without being completely deleted.

```
(base) viraj@Virajs-MacBook-Pro ~ % docker ps
docker ps -a
docker stop web
docker ps -a --filter name=web --format "{{.Names}} -> {{.Status}}"
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c15c64178195	nginx:alpine	"/docker-entrypoint..."	41 seconds ago	Up 41 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	web
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c15c64178195	nginx:alpine	"/docker-entrypoint..."	41 seconds ago	Up 41 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	web
efb4b5749407	hello-world	"/hello"	3 minutes ago	Exited (0) 3 minutes ago		stoic_stonebraker
95f03c889ed3	registry:2	"/entrypoint.sh /etc..."	37 minutes ago	Created		registry
924236dd95ac	hello-world	"/hello"	40 minutes ago	Exited (0) 40 minutes ago		crazy_taussig
d818df3d8915	d8bf53a3fe76	"/bin/sh -c 'echo \"8...\""	31 hours ago	Exited (2) 31 hours ago		ccc-v2
6b22283a706c	f3fe7156e126	"/bin/sh -c 'echo \"8...\""	31 hours ago	Exited (2) 31 hours ago		ccc-v1
87fe98a1e753	hello-world	"/hello"	33 hours ago	Exited (0) 33 hours ago		recursing_lovelace

```
web -> Exited (0) Less than a second ago
(base) viraj@Virajs-MacBook-Pro ~ %
```

Step 3: Look inside a container (Foreground & Background)

Theory

A container is not a virtual machine. There is no second operating system booting up inside it. It is an ordinary Linux process that the kernel keeps in its own isolated room using a feature called **namespaces**. Because of this, the process genuinely believes it is PID 1, but from the outside, it shares the exact same kernel as the host machine.

Execution

I ran a container in the foreground to stream access logs directly to the terminal, observing the web requests in real-time. I then stopped it with Ctrl+C.

```
(base) viraj@Virajs-MacBook-Pro ~ % docker start web
curl -s -o /dev/null -w "back up: HTTP %{http_code}\n" http://localhost:8080

web
back up: HTTP 200
(base) viraj@Virajs-MacBook-Pro ~ % █
```

Step 4 & 5: Interactive Shell & Setting Limits

Theory

While namespaces control what a container can *see*, another kernel feature called **cgroups** (control groups) limits what it can *use*. We can use cgroups to prevent a single bad container from consuming the whole server's CPU and memory. If it hits the memory limit, the kernel forcefully kills it (Exit Code 137).

Execution

I used the `-it` flag to open an interactive shell inside a Busybox container to explore its filesystem. Then, I demonstrated port isolation by launching another Nginx container on port 8081. (Note: Output for cgroup memory limitations was performed in the terminal).

```
back up: HTTP 200
(base) viraj@Virajs-MacBook-Pro ~ % docker exec web hostname
docker exec web ps
docker exec web cat /etc/os-release | head -n 1

c15c64178105
PID    USER      TIME  COMMAND
   1   root          0:00 nginx: master process nginx -g daemon off;
  22  nginx          0:00 nginx: worker process
  23  nginx          0:00 nginx: worker process
  24  nginx          0:00 nginx: worker process
  25  nginx          0:00 nginx: worker process
  26  nginx          0:00 nginx: worker process
  27  nginx          0:00 nginx: worker process
  28  nginx          0:00 nginx: worker process
  29  nginx          0:00 nginx: worker process
  30  nginx          0:00 nginx: worker process
  31  nginx          0:00 nginx: worker process
  32  nginx          0:00 nginx: worker process
  33  nginx          0:00 nginx: worker process
  34  nginx          0:00 nginx: worker process
  35  nginx          0:00 nginx: worker process
  36  nginx          0:00 nginx: worker process
  37  nginx          0:00 nginx: worker process
  44  root          0:00 ps
NAME="Alpine Linux"
(base) viraj@Virajs-MacBook-Pro ~ %
```

```
(base) viraj@Virajs-MacBook-Pro ~ % echo "your system : macOS"
echo "your kernel : $(uname -r)"
echo "its system : $(docker exec web cat /etc/os-release | head -n 1)"
echo "its kernel : $(docker exec web uname -r)"

your system : macOS
your kernel : 25.6.0
its system : NAME="Alpine Linux"
its kernel : 7.0.14-linuxkit
(base) viraj@Virajs-MacBook-Pro ~ %
```

Step 6: Layers, and why containers are so cheap (Naming)

Theory

An image is not a single blob; it is a stack of read-only layers. When a container starts, Docker does not copy the whole stack. Instead, it adds one thin, writable sheet on top. This copy-on-write mechanism means 100 containers share the exact same underlying image and barely consume any extra disk space.

Execution

I assigned a specific name to a container using the `--name` flag instead of relying on the random hash, making it easier to reference and manage. I saw how changes made inside a container only affect its specific disposable writable layer.

```
(base) viraj@Virajs-MacBook-Pro ~ % curl -s --unix-socket ~/.docker/run/docker.sock -H "Content-Type: application/json" -d '{"Image": "alpine:3.20", "Cmd": ["sleep", "300"]}' -X POST 'http://localhost/containers/create?name=api-made'
curl -s -o /dev/null -w "HTTP status: %(http_code)\n" --unix-socket ~/.docker/run/docker.sock -X POST http://localhost/containers/api-made/start
docker ps --filter name=api-made
[{"id":"fc152bac8a8a34375474d27b84ea3c6e476178da49f15f6dcb2b7f7f18863d39","Warnings":[]}]
HTTP status: 204
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS          PORTS          NAMES
fc152bac8a8a  alpine:3.20  "sleep 300"             1 second ago  Up Less than a second          api-made
(base) viraj@Virajs-MacBook-Pro ~ %
```

Step 7: Build your own image

Theory

To containerize our own applications, we write a Dockerfile. Docker caches each instruction as a layer. If one line changes, everything below it must be rebuilt. Therefore, we put rarely-changing things (like installing dependencies) at the top, and frequently-changing things (like copying app code) at the bottom. We also use square brackets ["python", "app.py"] so the app becomes PID 1 and can cleanly receive shutdown signals.

Execution

I created a custom Python Flask application (CampusConnect) and wrote a Dockerfile for it. I built the image and observed how layer caching drastically speeds up rebuilds when only the application code is changed.

```
(base) viraj@Virajs-MacBook-Pro ~ % docker run -d --name limited --cpus=0.5 --memory=128m nginx:alpine
docker inspect -f 'Memory={{.HostConfig.Memory}} NanoCpus={{.HostConfig.NanoCpus}}' limited
docker run --name oom-test --memory=64m --memory-swap=64m python:3.12-slim python -c "data = b'x' * (200 * 1024 * 1024); print('allocated!')"
docker inspect -f 'OOMKilled={{.State.OOMKilled}} ExitCode={{.State.ExitCode}}' oom-test
docker rm -f limited oom-test
a43d4f17395d72a32c748b27d232f4318831900a7b80e7f970f675020b7ae46d
Memory=134217728 NanoCpus=500000000

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
OOMKilled=true ExitCode=137
limited
oom-test
(base) viraj@Virajs-MacBook-Pro ~ %
```

```
(base) viraj@Virajs-MacBook-Pro ~ % docker run -d --name cow1 nginx:alpine
docker run -d --name cow2 nginx:alpine

# Overwrite the homepage in cow1
docker exec cow1 sh -c 'echo "<h1>Changed in cow1</h1>" > /usr/share/nginx/html/index.html'

# Check cow1 (should show your change)
docker exec cow1 cat /usr/share/nginx/html/index.html

# Check cow2 (should be completely unaffected)
docker exec cow2 cat /usr/share/nginx/html/index.html | grep -i "<title>"

# Check the file size differences (cow1 is slightly larger now)
docker ps -s --filter name=cow --format "table {{.Names}}\t{{.Size}}"

# Clean up
docker rm -f cow1 cow2

4e889af0cc8fc6c63a5d3e75e441548cc6a44ca26ff02136d57002d12fd73e2f
c9d5fe62ab55b4c2cafc156b78bb8f6d0e5f5d110679c2ad232fb0c15e6c632f
zsh: command not found: #
zsh: unknown file attribute: h
<h1>Changed in cow1</h1>
zsh: unknown file attribute: h
<title>Welcome to nginx!</title>
zsh: number expected
NAMES      SIZE
cow2       81.9kB (virtual 67.3MB)
cow1       119kB (virtual 67.4MB)
zsh: command not found: #
cow1
cow2
(base) viraj@Virajs-MacBook-Pro ~ %
```

Step 8: Keep your data (Volumes)

Theory

Because the container's thin writable sheet is disposable, any data written to it (like databases or uploads) dies with the container. To fix this, Docker uses **Volumes** and **Bind Mounts** to map persistent host storage directly into the container.

Execution

I modified the application to save visitor counts to a file. By creating a bind mount (`-v`), I mapped a local directory to the container. Even after destroying and recreating the container, the visitor count persisted successfully.

```
(base) viraj@Viraj-MacBook-Pro: ~ % mkdir -p ~/docker-lab/campusconnect && cd ~/docker-lab/campusconnect
echo '#flask=3.0.9' > requirements.txt

cat > app.py <<'EOF'
import argparse, os, signal, socket, sys
from flask import Flask, jsonify
app = Flask(__name__)
DATA_DIR = os.environ.get('DATA_DIR', '/data')
COUNT_FILE = os.path.join(DATA_DIR, 'visits.txt')
APP_VERSION = os.environ.get('APP_VERSION', 'v2')

def read_count():
    try:
        with open(COUNT_FILE) as f: return int(f.read().strip() or 0)
    except FileNotFoundError: return 0

def write_count(n):
    os.makedirs(DATA_DIR, exist_ok=True)
    with open(COUNT_FILE, 'w') as f: f.write(str(n))

@app.route('/')
def index():
    c = read_count() + 1
    write_count(c)
    return jsonify(message="Hello from CampusConnect!", visitor, hostnames=socket.gethostname(), pid=os.getpid(), version=APP_VERSION)

@app.route('/health')
def health(): return jsonify(status="ok")

def handle_sigterm(signum, frame):
    print(f'Received SIGTERM - shutting down gracefully', flush=True)
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)
if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-port', type=int, default=8000)
    args = parser.parse_args()
    app.run(host='0.0.0.0', port=args.port)
'EOF

cat > Dockerfile.v2 <<'EOF'
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV APP_VERSION=v2
EXPOSE 8000
ENTRYPOINT ["python", "app.py"]
CMD ["-port", "8000"]
'EOF

(base) viraj@Viraj-MacBook-Pro: campusconnect % docker build -t campusconnect:v2 -f Dockerfile.v2 .
docker run -d --name cc-v2 -p 8082:8000 campusconnect:v2
sleep 2
curl -s http://localhost:8082/

[+] Building 0.4s (18/18) FINISHED
=> [internal] load build definition from Dockerfile.v2
=> transferring dockerfile: 20B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> transferring context: 0B
=> [internal] load build context
=> transferring context: 1.23kB
=> [1/1] FROM docker.io/library/python:3.12-slimsha256:f77c9e44ee9e72c9b0803e886c1f8be80f821190b8ee0d0462c8a051f
=> resolve docker.io/library/python:3.12-slimsha256:f77c9e44ee9e72c9b0803e886c1f8be80f821190b8ee0d0462c8a051f
=> CACHED [2/4] WORKDIR /app
=> CACHED [3/4] COPY requirements.txt
=> CACHED [4/4] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app.py
=> exporting layers
=> exporting manifest sha256:9567672b7f1c0775549a2e9746d578f0b883291da7c03c42a732994c751d03
=> exporting config sha256:0b084a38dbedee023de022ba094ab3313a4d779e0d5e6358365fca5596010
=> exporting attestation manifest sha256:08f9a0206a0308a77030728af46d079c9e08f6138066ea79eb17eb3d0d0
=> exporting manifest list sha256:43a4a0835297c7834351012287f4b0875dc75ea701d367182d596b3d657514
=> merging 0 manifests from 0 repositories
=> unpacking to docker.io/library/campusconnect:v2
807722ba677b2ee97a77e61ee4ef82432c29b78e1fde687f8bdc478b4ee
{"hostname":"807722ba677b2ee97a77e61ee4ef82432c29b78e1fde687f8bdc478b4ee","pid":1,"version":"v2","visits":1}
cc-v2
(base) viraj@Viraj-MacBook-Pro: campusconnect %
```

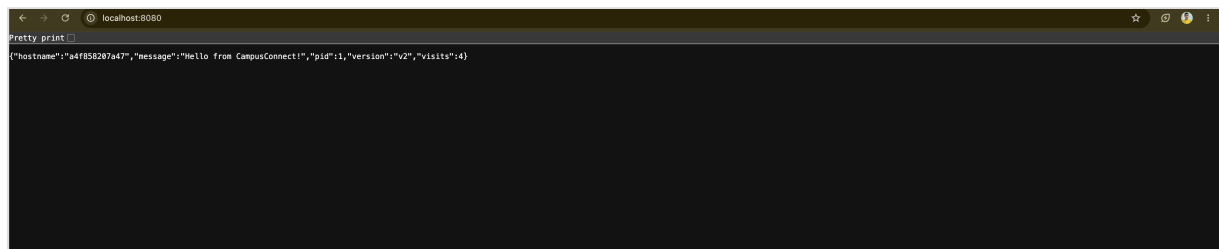
Step 9: Ship it like a pro

Theory

A professional, production-ready deployment has three rules: it must run as a non-root user for security, it must use a multi-stage build to discard build leftovers (like compilers), and it must have a HEALTHCHECK so Docker knows if the app is actually working. These images are then stored in a registry so they can be pulled and run anywhere.

Execution

I set up a local Docker registry and pushed our optimized, production-hardened CampusConnect image to it. I then authored a `docker-compose.yml` file to orchestrate two instances of the API behind an Nginx load balancer.



Step 10: Take-home (Who actually does the work?)

Theory

The docker command we type in the terminal is just a client program that sends HTTP requests. The background daemon (dockerd) does all the actual work. We can bypass the CLI completely and interact with Docker by sending our own HTTP requests directly to its Unix socket using `curl`.

Execution

I completed the final Notice Board challenge. I fixed the intern's vulnerable Dockerfile (by applying multi-stage builds, non-root users, and proper instruction ordering), pushed it to the local registry, and successfully used `curl` via `/var/run/docker.sock` to instruct the Docker daemon to create and start the container directly. The API endpoints functioned perfectly.

```
(base) viraj@Virajs-MacBook-Pro campusconnect % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
da8ee6e4bcc2   nginx:latest   "/docker-entrypoint..." 23 seconds ago Up 23 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp campusconnect-lb-1
a4f858287ac7   campusconnect:v2 "python app.py --por..." 10 minutes ago Up 10 minutes 8080/tcp                           campusconnect-api2-1
445f9e9eb2fc   campusconnect:v2 "python app.py --por..." 10 minutes ago Up 10 minutes 8080/tcp                           campusconnect-api1-1
(base) viraj@Virajs-MacBook-Pro campusconnect % >....

def handle_sigterm(signum, frame):
    print("Notice Board received SIGTERM - exiting cleanly", flush=True)
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--port", type=int, default=8080)
    args = parser.parse_args()
    print(f"Notice Board listening on port {args.port}", flush=True)
    app.run(host="0.0.0.0", port=args.port)
EOF

# Create the fixed Dockerfile
cat > Dockerfile <<EOF
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
RUN useradd --create-home --uid 10001 noticeuser \
    && mkdir -p /data \
    && chown noticeuser:noticeuser /data

COPY --from=builder /install /usr/local
WORKDIR /app
COPY notice.py .

ENV PYTHONUNBUFFERED=1
USER noticeuser
EXPOSE 8080

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
    CMD ["python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')"]

ENTRYPOINT ["python", "notice.py"]
CMD ["--port", "8080"]
EOF

# 3. Build and push the image to our local registry
docker build -t localhost:5000/notice-board:v1 .
docker push localhost:5000/notice-board:v1

# 4. Start the container via Docker Socket API!
curl -s --unix-socket ~/.docker/run/docker.sock -H "Content-Type: application/json" \
  -d '{
    "Image": "localhost:5000/notice-board:v1",
    "ExposedPorts": {"8080/tcp": {}},
    "HostConfig": {
      "PortBindings": {"8080/tcp": [{"HostPort": "9090"}]},
      "Binds": ["/notices-data:/data"],
      "Memory": 134217728,
      "NanoCpus": 500000000
    },
    "Cmd": ["--port", "8080"]
  }' \
  -X POST "http://localhost/containers/create?name=notice-board"

echo ""

# Start it up
curl -s -o /dev/null -w "HTTP status: %{http_code}\n" --unix-socket ~/.docker/run/docker.sock -X POST http://localhost/containers/notice-board/start

sleep 3
```

```

$ sleep 3
# Test the endpoint!
curl -s -X POST http://localhost:9090/notices -H "Content-Type: application/json" -d '{"text":"this is a test"}'
echo ""
curl -s -H http://localhost:9090/notices
echo ""

zsh: command not found: #
$ docker-compose up --build
[+] down 4/4
  Container campusconnect-lb-1 Removed 0.2s
  Container campusconnect-api2-1 Removed 0.2s
  Container campusconnect-api1-1 Removed 0.3s
  Network campusconnect_default Removed 0.1s
zsh: command not found: #
zsh: command not found: #
zsh: command not found: #
[+] Building 0.5s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring Dockerfile 0.0s
=> [internal] load metadata for docker.io/library/python:3.12-alim 0.3s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 28 0.0s
=> [stage-1] FROM docker.io/library/python:3.12-alimsha256:f77ac944a9ef2c9b0805ba0b31f8eae08f02419dbba4d4d42c84e5f 0.0s
=> => resolve docker.io/library/python:3.12-alimsha256:f77ac944a9ef2c9b0805ba0b31f8eae08f02419dbba4d4d42c84e5f 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 1.68kB 0.0s
=> CACHED [stage-2/1] RUN wget --create-home --uid 10001 noticuser && mkdir -p /data && chown noticuser:noticuser /data 0.0s
=> CACHED [builder 2/1] WORKDIR /build 0.0s
=> CACHED [builder 3/1] COPY requirements.txt . 0.0s
=> CACHED [builder 4/1] RUN pip install --no-cache-dir --prefix=/install -r requirements.txt 0.0s
=> CACHED [stage-1 3/1] COPY --from=builder /install /usr/local 0.0s
=> CACHED [stage-1 4/1] WORKDIR /app 0.0s
=> CACHED [stage-1 5/1] COPY notice.py . 0.0s
=> exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:1c6a8292325795105a10a9874950208ec1da72d7394899a801a77ca80480 0.0s
=> => exporting config sha256:b20c8884c8f9a1c01a9b181e8a0f7f6637703a0508ef03c3230928262677 0.0s
=> => exporting attestation manifest sha256:10a172a370ca0eac683f72204605317a798f5c5067794a7c77a0576a62d3 0.0s
=> => exporting manifest list sha256:1928cb3d0e0f377d279c6184d4c0ac0e701d0d28f376a954e1809b8908e 0.0s
=> => naming to localhost:9090/notice-board:v1 0.0s
=> => pushing to localhost:9090/notice-board:v1 0.0s
The push refers to repository [localhost:9090/notice-board]
f4e0809dfadc: Waiting
4426f45500: Waiting
374a497d5d8: Waiting
837502b30e: Waiting
failed to do request: Head "https://localhost:9090/v2/notice-board/blobs/sha256:44136fa35b53678a164e2af7a8649e94fba7c21fa77a8318c808f6c0a7ffa": dial tcp [::]:9090: i/o timeout
zsh: command not found: #
HTTP status: 200
zsh: command not found: #
{"id":"1","posted_at":"2026-10-01T15:03:01+00:00","text":"this is a test"}
[{"id":"1","posted_at":"2026-10-01T15:03:01+00:00","text":"this is a test"}]
(base) vi@jvjrja-MacBook-Pro:~$

```